

在线聊天系统开发文档

作品名称：在线聊天系统

小组名称：#1f1e33

1.成员分工及权重：

1.1 杨睿涵（通信模块负责人，权重1.09）

核心职责：WebSocket通信架构设计与实现

具体贡献：

- WebSocket核心架构设计：ChatHandler、WebSocketConfig、AuthHandshakeInterceptor
- HandlerRegistry注册调度机制：实现策略模式+注册表模式，支持Handler自动注册与路由分发
- AC自动机算法实现：设计并实现SensitiveWordFilter，完成敏感词过滤功能，算法复杂度O(n+m)
- BaseActionHandler基类设计：提供统一错误处理与响应格式，应用模板方法模式
- 线程池配置与异步消息处理：AppConfig中配置ThreadPoolTaskExecutor，优化并发性能
- API文档v1.0/v2.0制定：主导WebSocket消息格式与接口协议设计
- 代码重构与架构优化：统一异常处理机制，优化HandlerRegistry自动注册逻辑

1.2 马绍博（前端模块负责人，权重0.97）

核心职责：前端UI设计与交互实现

具体贡献：

- UI界面设计：仿微信风格界面，使用Flexbox布局实现响应式设计
- WebSocket客户端连接管理：实现连接建立、断开重连、心跳检测机制
- 消息收发界面与交互：消息渲染、输入框处理、自动滚动等功能
- 好友列表与群组列表展示：动态更新、状态显示、交互操作
- 在线状态实时更新：监听WebSocket事件，实时同步用户上下线状态
- @提醒UI展示：识别@符号、显示提醒徽章、点击清除提醒
- 消息撤回与反应UI交互：撤回消息删除、反应按钮与计数显示
- 前端Mock数据设计：支持前后端并行开发

1.3 朱律霖（消息模块负责人，权重0.97）

核心职责：消息业务逻辑实现

具体贡献：

- MessageService实现：消息处理、保存、查询逻辑
- 消息撤回功能：实现2分钟时间限制与权限检查，支持多端同步
- 消息反应功能：点赞、点踩等反应类型，实时更新反应计数
- 历史消息查询：支持分页查询，按时间排序，游标翻页机制

- @提醒功能后端实现：解析atUsers列表，推送提醒事件
- 相关Handler实现：Send_PrivateHandler、Send_GroupHandler、Recall_MsgHandler、Msg_ReactHandler、Get_HistoryHandler
- UML架构图制作：绘制系统架构图，展示四层分层架构与模块关系
- 消息模块测试用例设计：覆盖单聊、群聊、撤回、反应等场景

1.4 赵琦（用户模块负责人，权重0.97）

核心职责：用户业务与权限管理

具体贡献：

- UserService接口定义与实现：登录、登出、用户注册、密码验证
- 好友管理功能：发送/接受/拒绝/删除好友申请，好友列表维护
- 群组管理功能：创建、加入、退出、解散群组，群成员管理
- 权限管理实现：管理员、群主、群管理员三级权限体系，权限检查逻辑
- 管理员全局功能：全局禁言、强制踢出用户功能实现
- 相关Handler实现：LoginHandler、Send_FriendRequestHandler、Accept_FriendRequestHandler、Create_GroupHandler、Mute_UserHandler、Kick_UserHandler等15+个Handler
- 权限测试用例设计：覆盖管理员、群主、群管理员各类权限场景

2.项目介绍

2.1 项目简介

本系统为基于Spring Boot 3.3.0与Java 21的轻量级即时通讯平台，采用前后端分离架构。后端利用现代Java的并发特性，通过WebSocket协议实现低延迟、高并发的全双工通信。前端采用原生Web技术栈（HTML5/CSS3/JavaScript）构建，无框架依赖。系统完整实现了即时通讯的核心功能，覆盖单聊、群聊、好友管理、权限控制等模块。

2.2 开发目的

实践WebSocket全双工通信，解决HTTP轮询的实时性瓶颈；应用设计模式（HandlerRegistry策略模式）提升系统可扩展性；探索AC自动机算法在敏感词过滤场景的高效应用。

2.3 技术栈选型

2.3.1 后端技术栈

技术	版本	选型理由
Java	21 (LTS)	LTS版本，性能优化，现代语法特性
Spring Boot	3.3.0	自动配置，快速开发，生态完善
WebSocket	Spring WebSocket	全双工通信，低延迟推送
Jackson	3.x	JSON序列化，框架默认集成

技术	版本	选型理由
Lombok	最新	代码生成，减少样板代码
Maven	3.8+	依赖管理与构建工具

2.3.2 前端技术栈

技术	说明	选型理由
HTML5	语义化标签	原生支持，零依赖
CSS3	Flexbox布局	现代布局，响应式设计
原生JavaScript	ES6+	无框架依赖，轻量高效
WebSocket API	浏览器原生API	标准WebSocket客户端实现

2.3.3 数据存储

采用ConcurrentHashMap实现内存存储，具备以下特点：

- **优势：**零配置，读写性能优异，无I/O瓶颈
- **适用场景：**演示项目，快速验证功能
- **限制：**进程重启后数据丢失，不支持持久化

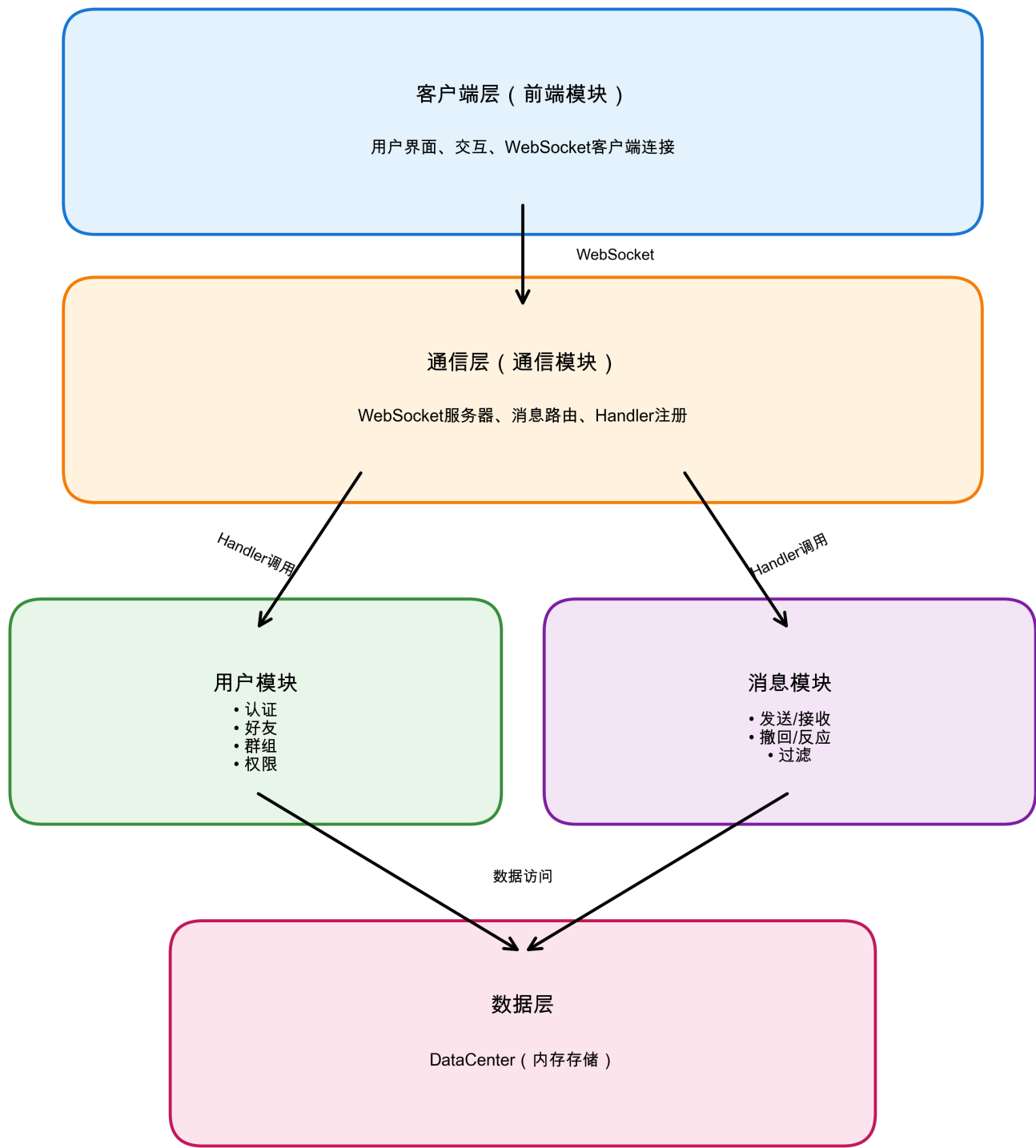
2.3.4 开发工具

- **IDE：**IntelliJ IDEA / VS Code
- **版本控制：**Git + GitHub
- **构建工具：**Maven
- **测试工具：**浏览器开发者工具、Postman

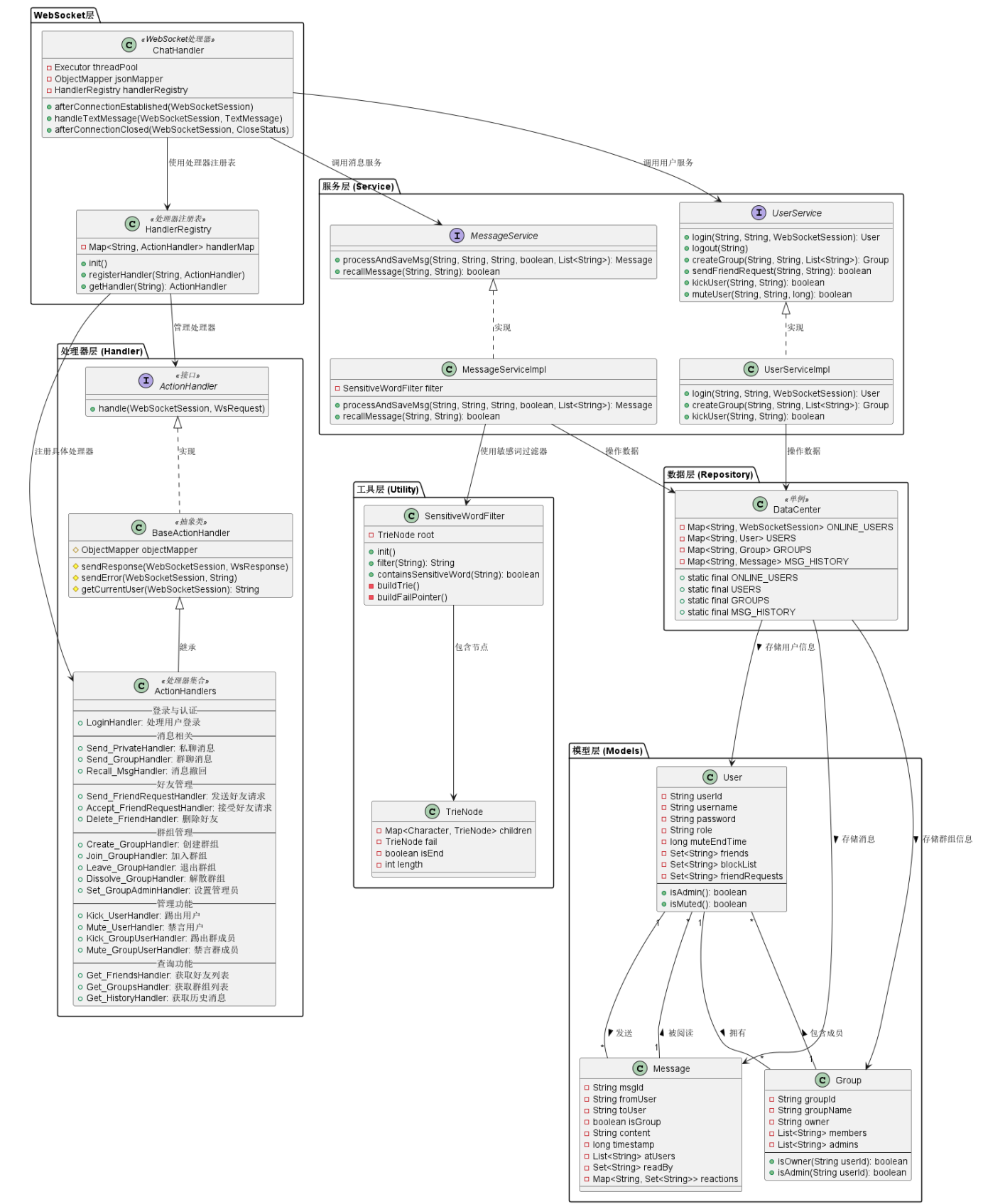
2.4 系统架构设计

2.4.1 四层分层架构

实时聊天系统 - 架构图



如下为UML图：



2.4.2 模块职责划分

客户端层（前端模块 - 马绍博负责）：

- 用户界面展示和交互
- WebSocket客户端连接管理
- 消息渲染和状态管理

通信层（通信模块 - 杨睿涵负责）：

- WebSocket服务器配置和管理
- 消息路由分发（HandlerRegistry）
- Handler自动注册和调度

业务层：

- **用户模块（赵琦负责）：** 用户认证、好友管理、群组管理、权限控制
- **消息模块（朱律霖负责）：** 消息发送、撤回、反应、历史查询、敏感词过滤

数据层：

- 统一数据存储（DataCenter）
- 线程安全的数据结构（ConcurrentHashMap）

2.4.3 设计原则

1. **单一职责原则：** 模块与Handler职责单一，边界清晰
2. **开闭原则：** 通过HandlerRegistry模式实现扩展开放、修改封闭
3. **依赖倒置原则：** 依赖ActionHandler抽象接口，而非具体实现
4. **接口隔离原则：** Service接口粒度合理，避免冗余方法

3.团队协作方式描述

3.1 项目进度规划

开发周期：**2025年11月25日 - 2025年12月24日**（4周）

3.1.1 第一阶段：需求分析与架构设计（11月25日 - 11月30日）

目标： 完成架构设计、技术选型与接口定义。

主要工作：

1. **需求分析（全体成员）：**
 - 调研主流IM系统功能特性
 - 分析WebSocket在Spring Boot 3中的实践
 - 明确功能模块边界
2. **技术选型（杨睿涵主导）：**
 - Spring Boot 3.3.0 + Java 21
 - WebSocket作为通信协议
 - ConcurrentHashMap实现内存存储
3. **架构设计（全体讨论）：**
 - 四层架构：客户端层、通信层、业务层、数据层
 - 模块划分：通信、前端、消息、用户四大模块

- 核心机制：HandlerRegistry路由模式

4. 接口定义（模块负责人）：

- **UserService**接口（赵琦）
- **MessageService**接口（朱律霖）
- WebSocket消息格式与API协议（杨睿涵）

产出物：系统架构图、API文档v1.0、初始代码框架

3.1.2 第二阶段：并行开发与API对接（12月1日 - 12月15日）

目标：基于API文档实现各模块并行开发与前后端对接。

工作分配与进度：

杨睿涵（通信模块） - Week 2-3：

- 实现WebSocket核心架构：**ChatHandler**（消息入口与路由）、**WebSocketConfig**（WebSocket配置）、**AuthHandshakeInterceptor**（握手拦截器）
- 实现HandlerRegistry注册调度机制：策略模式+注册表模式，支持Handler自动注册与路由分发
- 实现BaseActionHandler基类：模板方法模式，提供统一错误处理与响应格式
- 实现**AC自动机算法**：设计**SensitiveWordFilter**类，实现敏感词过滤，算法复杂度 $O(n+m)$ ，过滤时间 $<1ms$
- 配置线程池：**AppConfig**中配置**ThreadPoolTaskExecutor**，实现异步消息处理，提升并发性能
- 制定API文档v1.0：定义WebSocket消息格式与接口协议，为前后端协作提供规范

赵琦（用户模块） - Week 2-3：

- 实现**UserServiceImpl**：登录、登出、用户注册、密码验证逻辑
- 实现好友管理：发送/接受/拒绝/删除好友申请，好友列表维护
- 实现群组管理：创建、加入、退出、解散群组，群成员管理
- 实现权限管理：三级权限体系（管理员、群主、群管理员），权限检查逻辑设计
- 实现管理员全局功能：全局禁言、强制踢出用户功能
- 实现相关Handler：**LoginHandler**、**Send_FriendRequestHandler**、**Accept_FriendRequestHandler**、**Create_GroupHandler**、**Mute_UserHandler**、**Kick_UserHandler**等15+个Handler
- 设计权限检查责任链：Handler层快速检查+Service层详细验证

朱律霖（消息模块） - Week 2-3：

- 实现**MessageServiceImpl**：消息处理、保存、查询逻辑，集成敏感词过滤
- 实现消息撤回功能：2分钟时间限制，权限检查，多端同步机制
- 实现消息反应功能：点赞、点踩等反应类型，实时更新反应计数
- 实现历史消息查询：分页查询机制，按时间排序，游标翻页实现
- 实现@提醒功能后端：解析atUsers列表，推送提醒事件
- 实现相关Handler：**Send_PrivateHandler**、**Send_GroupHandler**、**Recall_MsgHandler**、**Msg_ReactHandler**、**Get_HistoryHandler**
- 制作**UML架构图**：绘制系统四层分层架构图，展示模块职责与调用关系
- 设计消息模块测试用例：覆盖单聊、群聊、撤回、反应等场景

马绍博（前端模块） - Week 2-3:

- 设计并实现UI界面：仿微信风格，Flexbox布局，响应式设计
- 实现WebSocket客户端连接管理：连接建立、断开重连、心跳检测机制（30秒间隔）
- 实现消息收发界面和交互：消息渲染、输入框处理、自动滚动、消息状态显示
- 实现好友列表与群组列表展示：动态更新、状态显示、交互操作（点击切换聊天对象）
- 实现在线状态实时更新：监听WebSocket事件，实时同步用户上下线状态
- 实现@提醒UI展示：识别@符号、显示提醒徽章、点击清除提醒
- 实现消息撤回与反应UI交互：撤回消息删除、反应按钮与计数显示
- 设计前端Mock数据：支持前后端并行开发，使用模拟数据测试UI

协作机制：

1. API文档驱动：

- 初期由通信、用户、消息模块负责人协作制定完整API文档
- 统一消息格式：请求{**action**, **params**}，响应{**type**, **code**, **msg**, **data**}

2. 并行开发：

- 后端基于API文档实现Handler与Service
- 前端基于API文档实现消息收发逻辑
- 双方按约定格式对接

3. 测试联调：

- 功能完成后通过Postman或浏览器控制台测试
- 问题及时沟通，必要时迭代更新API文档

3.1.3 第三阶段：功能迭代与优化（12月16日 - 12月21日）

目标：代码重构、Bug修复与性能优化。

迭代背景：模块独立开发导致代码冗余，异常处理不统一，存在并发安全隐患。

迭代内容：

1. 代码重构（杨睿涵主导）：

- 统一所有Handler继承**BaseActionHandler**基类，应用模板方法模式
- 统一异常处理与错误响应格式，使用**WsResponse.error()**方法
- 优化HandlerRegistry自动注册逻辑，改进Bean名称解析规则

2. 功能完善（各模块负责人）：

- **用户模块（赵琦）**：完善权限校验逻辑，新增管理员全局禁言/踢人功能，实现**Mute_UserHandler**和**Kick_UserHandler**
- **消息模块（朱律霖）**：优化群聊消息广播性能，使用异步批量发送，解决大群（100+人）消息延迟问题
- **前端模块（马绍博）**：优化UI交互体验，完善加载状态提示，修复消息列表滚动问题，改进@提醒显示效果
- **通信模块（杨睿涵）**：优化线程池配置参数，增加WebSocket连接心跳检测机制

3. **Bug修复**（各模块协作）：

- 修复多端登录Session管理问题（赵琦）
- 修复消息撤回后UI同步问题（朱律霖+马绍博协作）
- 修复@提醒在某些场景下的显示异常（朱律霖+马绍博协作）
- 修复历史消息分页边界处理问题（朱律霖）

4. **性能优化**（杨睿涵主导）：

- 加入AC自动机过滤算法
- 群聊消息异步批量发送，使用线程池并行处理
- 前端消息列表渲染优化（考虑虚拟滚动方案，马绍博实现）

迭代成果：代码质量显著提升，系统稳定性增强，用户体验优化。

3.1.4 **第四阶段：测试与文档完善（12月22日 - 12月24日）**

目标：全面测试、文档编写与答辩准备。

测试工作：

- 功能测试：覆盖所有核心功能点
- 压力测试：多用户并发场景验证
- 兼容性测试：主流浏览器兼容性验证

文档编写：

- 完成API文档v2.0（杨睿涵主导，团队协作）
- 编写使用文档与部署文档（各模块负责人分别编写）
- 制作UML架构图（朱律霖）
- 准备答辩材料（全体成员）

3.2 API文档驱动的协作方式

3.2.1 API文档的设计与制定

项目初期（11月28日-11月30日），由通信模块负责人牵头，联合用户模块与消息模块负责人，协作完成API接口文档v1.0。

API文档结构：

```
# 聊天室系统 - API开发文档 (v2.0)

## 1. WebSocket连接配置
- URL: ws://{server_ip}:8080/chat
- 协议: JSON格式
- 时间戳: 13位毫秒级Unix Timestamp

## 2. 数据包基础结构

客户端请求格式:
```

```
{
  "action": "指令类型 (String)",
  "params": { ... }
}
```

服务端响应格式：

```
{
  "type": "事件类型 (String)",
  "code": 200,
  "msg": "ok",
  "data": { ... }
}
```

3. 接口列表（示例）

登录接口

Action: LOGIN

Params: { "username": "Tom", "password": "123" }

Response: { "type": "LOGIN_RESP", "code": 200, "data": {...} }

发送私聊

Action: SEND_PRIVATE

Params: { "targetUser": "Bob", "content": "Hello" }

Response: { "type": "EVENT_CHAT_MSG", "data": {...} }

...

文档特点：

- 统一的请求/响应格式规范
- 参数类型与必填项明确标注
- 响应类型(type字段)清晰定义
- 版本化管理，迭代同步更新

3.2.2 前后端基于API文档的协作流程

第一阶段：接口设计（11月28-30日）

1. 需求讨论（全体成员）：

- 确定需要哪些功能接口
- 讨论每个接口的参数和返回值
- 确定错误处理和异常情况的响应格式

2. 文档编写（杨睿涵主导）：

- 编写完整的API文档v1.0
- 包含所有接口的详细说明
- 提供请求/响应示例

3. 文档评审（全体成员）：

- 所有成员review API文档

- 提出修改建议
- 确认最终版本

第二阶段：并行开发（12月1-15日）

后端开发流程：

1. 查看API文档，了解接口要求
↓
2. 实现Service接口（UserService、MessageService）
↓
3. 实现Handler处理器（LoginHandler、Send_PrivateHandler等）
↓
4. 按照API文档格式构造响应
↓
5. 本地测试，确保响应格式符合API文档

前端开发流程：

1. 查看API文档，了解接口格式
↓
2. 编写Mock数据，模拟后端响应
↓
3. 实现UI界面和交互逻辑
↓
4. 实现消息发送逻辑（按照API文档格式）
↓
5. 实现消息接收处理（按照API文档的响应格式）
↓
6. 使用Mock数据测试UI和交互

协作示例：发送私聊消息功能

API文档约定：

```
// 请求
{
  "action": "SEND_PRIVATE",
  "params": {
    "targetUser": "Bob",
    "content": "Hello"
  }
}

// 响应
{
  "type": "EVENT_CHAT_MSG",
  "code": 200,
```

```
"data": {
  "msgId": "uuid",
  "fromUser": "Alice",
  "toUser": "Bob",
  "content": "Hello",
  "timestamp": 1234567890123
}
```

后端实现（朱律霖）：

```
// Send_PrivateHandler.java
@Override
public void handle(WebSocketSession session, WsRequest request) {
    // 1. 解析参数（按照API文档格式）
    String targetUser = params.get("targetUser").asText();
    String content = params.get("content").asText();

    // 2. 处理业务逻辑
    Message message = messageService.processAndSaveMsg(...);

    // 3. 构造响应（严格按照API文档格式）
    WsResponse response = WsResponse.builder()
        .type("EVENT_CHAT_MSG") // 按文档约定
        .code(200)
        .data(message)
        .build();

    // 4. 发送响应
    sendResponse(session, response);
}
```

前端实现（马绍博）：

```
// app.js
// 1. 发送消息（按照API文档格式）
sendMessage() {
    app.send({
        action: 'SEND_PRIVATE', // 按文档约定
        params: {
            targetUser: this.currentTarget.id, // 按文档格式
            content: document.getElementById('messageInput').value
        }
    });
}

// 2. 接收消息（按照API文档的响应格式）
handleServerMessage(response) {
    switch (response.type) { // 按文档约定的type字段
```

```
        case 'EVENT_CHAT_MSG': // 按文档约定
            // 处理消息
            this.handleChatMessage(response.data);
            break;
    }
}
```

优势：

- 前后端并行开发，无需相互等待
- 格式统一，降低对接错误率
- 前后端可独立测试（前端Mock数据，后端独立接口测试）

3.2.3 API文档的迭代与维护

第一次迭代（12月16-18日）：

迭代原因：

- 发现某些接口的响应格式需要优化
- 某些功能需要新增接口
- 错误处理格式需要统一

迭代过程：

1. 问题发现：

- 发现错误响应格式不统一，有些用msg字段，有些用message字段
- 发现某些场景下需要新增事件类型

2. 文档更新：

- 统一错误响应格式：{type: "ERROR", code: 500, msg: "错误信息"}
- 新增管理员操作相关的事件类型
- 更新API文档版本到v2.0

3. 代码同步更新：

- 后端统一使用WsResponse.error()方法
- 前端统一处理ERROR类型的响应
- 所有成员同步更新代码

文档管理：

- API文档使用Markdown格式，便于版本控制
- 使用Git管理文档版本
- 重要变更在团队群组中通知

3.2.4 API文档价值

降低沟通成本：统一接口规范，减少理解偏差与格式错误。

提升开发效率：支持前后端并行开发，缩短联调周期。

保障代码质量：接口格式规范化，便于测试与调试。

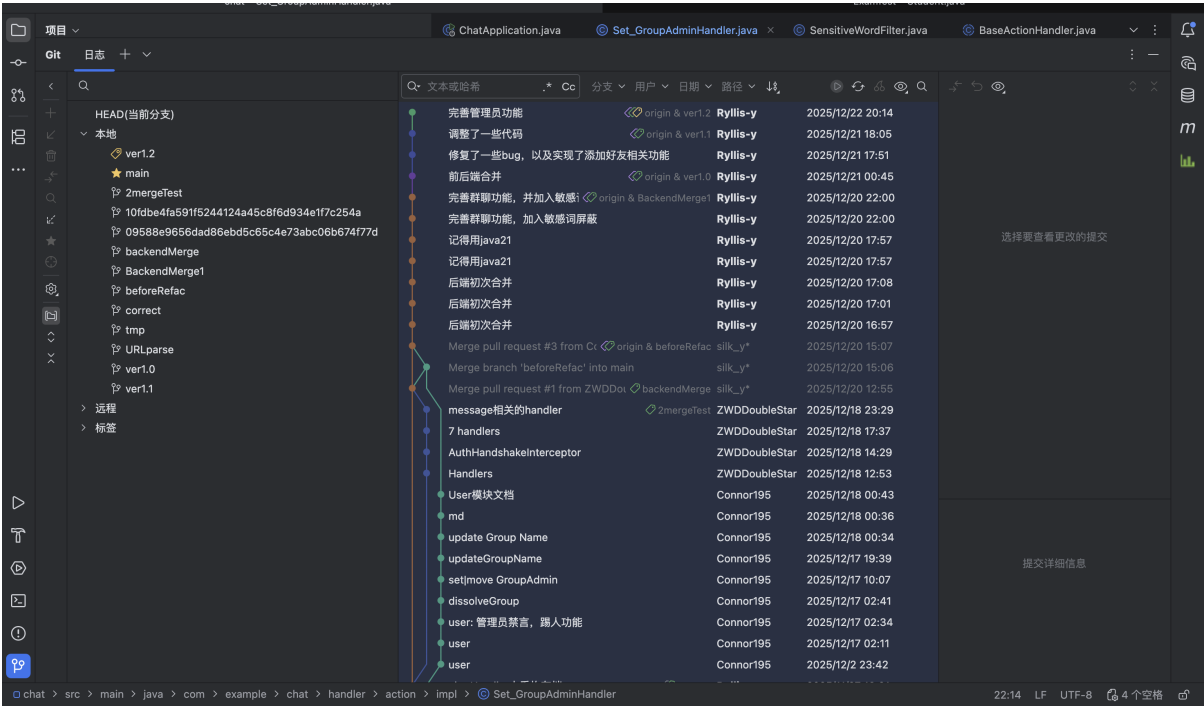
3.3 协同编码方式与工具链

3.2.1 代码托管与版本管理

GitHub工作流：

1. 分支策略：

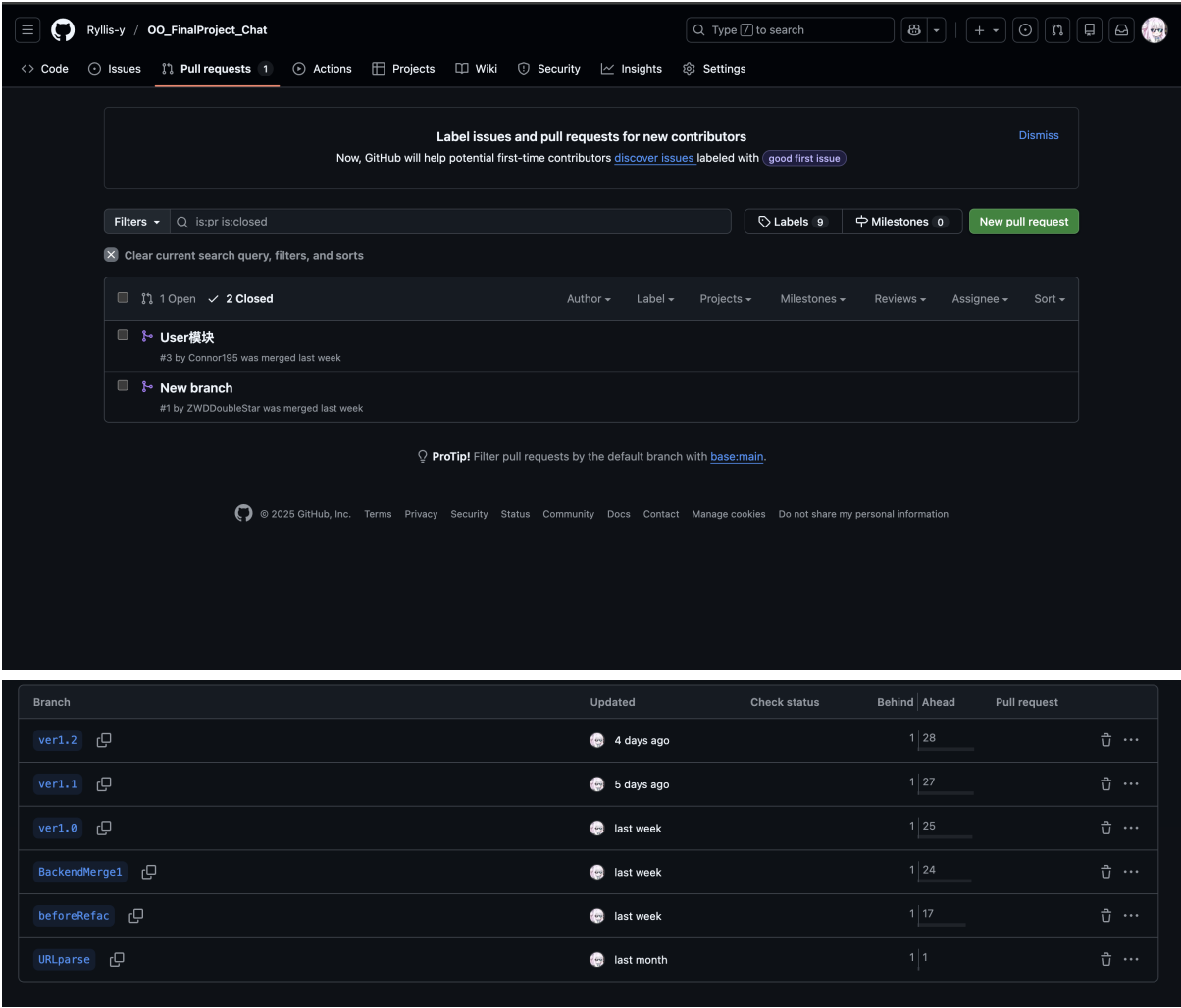
- 先上传基本的代码框架在main分支
- 每次进行修改时创建新分支以便回滚
- 初步完成后将项目上传到ver1.0分支，后续迭代时再创建后续的版本分支



2. 协作流程：

- 每个成员从develop分支创建自己的特性分支
- 完成功能后，提交Pull Request到develop分支

- 代码审查通过后处理冲突并合并，确保代码质量



3.2.2 前后端协作方式

API文档统一标准：

1. API文档结构：

```
## API接口文档v2.0

### 1. WebSocket连接
- URL：ws://localhost:8080/chat
- 协议：JSON格式

### 2. 消息格式
- 请求格式：{action: "ACTION", params: {...}}
- 响应格式：{type: "TYPE", code: 200, data: {...}}

### 3. 接口列表
| Action | 参数 | 说明 | 响应 |
|-----|-----|-----|-----|
| LOGIN | username, password | 登录 | LOGIN_RESP |
| SEND_PRIVATE | targetUser, content | 发送私聊 | EVENT_CHAT_MSG |
...
```

2. 前后端对接流程：



3. 问题解决机制：

- 发现API不匹配时，优先更新API文档
- 紧急情况下通过团队群组沟通
- 重大变更需要团队讨论通过

3.2.3 代码审查机制

Pull Request审查要点：

- 代码规范性：命名、格式、注释
- 功能正确性：逻辑是否正确，边界情况是否处理
- 性能考虑：是否存在性能问题
- 安全性：权限检查是否完善

审查流程：

1. 开发者提交PR，描述功能变更
2. 至少一名其他成员审查代码
3. 审查通过后合并，有问题则反馈修改

3.3 团队协作中的挑战与解决

3.3.1 前后端进度不同步

问题：初期后端接口未完成，前端无法完整测试；UI设计变更需后端调整响应格式。

解决方案：前端使用Mock数据先行开发；提前确定API文档，减少后期变更；问题及时沟通快速调整。

3.3.2 代码冲突与合并问题

问题：多人同时修改导致冲突；模块间接口变更产生连锁影响。

解决方案：模块化设计降低耦合；Service接口先行定义；接口变更及时同步团队。

3.3.3 功能迭代兼容性

问题：API格式变更需同步更新前后端；新功能可能与旧功能冲突。

解决方案：API文档版本化管理；新功能保持向后兼容；迭代后全面测试验证。

3.4 团队协作成果

代码统计（截至12月24日）：

- 后端Java代码：约3500行
- 前端JavaScript代码：约2500行
- HTML/CSS代码：约800行
- Handler处理器：25个
- API接口：30+个

协作效率：

- 模块化设计+API文档驱动，实现高效并行开发
- 代码审查机制保障代码质量
- 迭代开发确保功能稳定性与用户体验

4.设计模式应用

本项目在架构设计与代码实现中应用了多种经典设计模式，提升了系统的可扩展性、可维护性与代码复用性。

4.1 策略模式（Strategy Pattern）

应用场景：Handler路由机制

实现方式：

- 定义`ActionHandler`接口作为策略接口
- 每个具体Handler（如`LoginHandler`、`Send_PrivateHandler`）实现该接口
- `HandlerRegistry`作为上下文，根据action选择具体策略

优势：新增功能只需实现新的Handler，无需修改核心路由代码，符合开闭原则。

4.2 注册表模式（Registry Pattern）

应用场景：Handler自动注册与发现

实现方式：

- `HandlerRegistry`作为注册表，管理所有Handler实例
- 利用Spring的依赖注入自动发现所有`ActionHandler`实现
- `@PostConstruct`初始化时自动注册所有Handler

优势：实现Handler的自动发现与注册，降低维护成本。

4.3 模板方法模式（Template Method Pattern）

应用场景：BaseActionHandler基类设计

实现方式：

- `BaseActionHandler`定义模板方法：`sendResponse`、`sendError`、`sendSuccess`
- 所有具体Handler继承基类，复用公共方法
- 子类只需实现`handle`方法处理具体业务逻辑

优势：统一错误处理与响应格式，减少代码重复，25个Handler共享公共方法。

4.4 工厂模式 (Factory Pattern)

应用场景：Spring Bean创建与管理

实现方式：

- Spring容器作为工厂，通过@Component、@Service等注解创建Bean
- AppConfig中通过@Bean方法创建ObjectMapper、ThreadPoolTaskExecutor等对象
- 依赖注入机制实现对象的创建与依赖管理

优势：解耦对象创建与使用，便于测试与扩展。

应用框架：Spring Boot框架

4.5 单例模式 (Singleton Pattern)

应用场景：DataCenter数据存储中心

实现方式：

- DataCenter类使用静态final字段存储全局数据
- 所有数据存储（USERS、GROUPS、MSG_HISTORY等）为静态单例
- 通过@Component注解，Spring容器管理单例Bean

优势：全局唯一数据存储，保证数据一致性。

4.6 依赖注入模式 (Dependency Injection)

应用场景：组件间依赖管理

实现方式：

- 通过@Autowired注解实现依赖注入
- Handler中注入Service，Service中注入Repository
- 接口与实现分离，便于测试与替换

优势：降低耦合度，提高代码可测试性。

应用框架：Spring Boot框架

4.7 建造者模式 (Builder Pattern)

应用场景：WsResponse、Message等对象构建

实现方式：

- 使用Lombok的@Builder注解生成建造者代码
- 链式调用构建复杂对象，提高代码可读性

优势：简化对象构建，支持可选参数，代码更清晰。

应用工具：Lombok

4.8 观察者模式 (Observer Pattern)

应用场景：WebSocket事件推送

实现方式：

- 前端作为观察者，监听WebSocket消息事件
- 后端作为被观察者，推送各类事件（消息、状态更新、系统通知等）
- 通过事件类型（type字段）区分不同事件

优势：实现松耦合的事件通知机制，支持一对多通信。

4.9 适配器模式（Adapter Pattern）

应用场景：WebSocket消息格式转换

实现方式：

- `ChatHandler`作为适配器，将WebSocket消息转换为内部请求格式
- `ObjectMapper`作为适配器，实现JSON与Java对象之间的转换

优势：屏蔽底层协议差异，统一消息处理接口。

4.10 责任链模式（Chain of Responsibility Pattern）

应用场景：权限检查链

实现方式：

- Handler层进行快速权限检查（第一层）
- Service层进行详细业务权限验证（第二层）
- 两层检查形成责任链，逐层过滤非法请求

优势：分层权限验证，提高安全性，便于扩展。

设计模式统计：

- 核心设计模式：策略模式、注册表模式、模板方法模式
- 框架提供模式：工厂模式、单例模式、依赖注入
- 工具支持模式：建造者模式（Lombok）
- 通信模式：观察者模式、适配器模式
- 安全模式：责任链模式

设计模式应用价值：

- 提升代码可维护性：模块职责清晰，易于理解与修改
- 增强系统可扩展性：新增功能无需修改核心代码
- 提高代码复用性：公共逻辑抽取，减少重复代码
- 降低系统耦合度：接口与实现分离，依赖注入管理

5. 高级算法与出彩设计

5.1 核心路由架构（HandlerRegistry模式）

5.1.1 设计背景与问题

传统方式的痛点：

```
// 传统switch-case方式（不采用）
switch (action) {
    case "LOGIN":
        handleLogin(session, request);
        break;
    case "SEND_PRIVATE":
        handleSendPrivate(session, request);
        break;
    case "SEND_GROUP":
        handleSendGroup(session, request);
        break;
    // ... 随着功能增加，这个switch会越来越臃肿
    case "XXX": // 每次新增功能都要修改这个switch
        handleXXX(session, request);
        break;
}
```

问题：

- 核心路由代码臃肿（30+个case）
- 新增功能需修改核心代码，违反开闭原则
- 耦合度高，难以测试与维护

5.1.2 HandlerRegistry模式实现

核心设计思想：策略模式 + 注册表模式的组合

代码实现：

```
// 1. 策略接口定义
public interface ActionHandler {
    void handle(WebSocketSession session, WsRequest request);
}

// 2. 注册表实现
@Component
public class HandlerRegistry {
    private final Map<String, ActionHandler> handlerMap = new
    ConcurrentHashMap<>();

    // Spring自动注入所有实现了ActionHandler的Bean
    @Autowired(required = false)
    private Map<String, ActionHandler> actionHandlerBeans;

    @PostConstruct
    public void init() {
```

```
// 自动注册所有Handler
actionHandlerBeans.forEach((beanName, handler) -> {
    String action = extractActionFromBeanName(beanName);
    registerHandler(action, handler);
});
}

public ActionHandler getHandler(String action) {
    return handlerMap.get(action.toUpperCase());
}
}

// 3. 具体策略实现
@Component
public class LoginHandler extends BaseActionHandler {
    @Override
    public void handle(WebSocketSession session, WsRequest request) {
        // 登录业务逻辑
    }
}

// 4. 路由使用
ActionHandler handler = handlerRegistry.getHandler(request.getAction());
if (handler != null) {
    handler.handle(session, request);
}
```

设计优势：

- **零入侵扩展**：新增功能只需创建新的Handler类，无需修改核心路由代码
- **自动注册**：利用Spring的依赖注入自动发现和注册Handler
- **易于测试**：每个Handler可以独立测试
- **代码组织清晰**：一个功能一个Handler，职责单一

效果：

- 25+个Handler，**ChatHandler**核心路由仅约20行
- switch-case方案需100+行，维护成本高

5.2 高效敏感词过滤（AC自动机算法）

5.2.1 算法选择与性能对比

问题背景：

- 需同时匹配多个敏感词（当前20个，可扩展至数千）
- 消息发送为高频操作，过滤性能要求严格
- 过滤延迟需控制在用户无感范围

算法对比：

算法	时间复杂度	空间复杂度	适用场景
暴力匹配	$O(n \times m)$	$O(1)$	敏感词数量少 (<10个)
KMP单模式匹配	$O(n + m)$	$O(m)$	单个敏感词匹配
AC自动机	$O(n + m)$	$O(\Sigma \times m)$	多模式匹配

其中：

- n ：文本长度
- m ：所有敏感词总长度
- Σ ：字符集大小

5.2.2 AC自动机算法实现

核心数据结构：

```
// Trie树节点
private static class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>(); // 子节点
    TrieNode fail = null; // 失败指针 (AC自动机核心)
    boolean isEnd = false; // 是否为敏感词结尾
    int length = 0; // 敏感词长度
}
```

算法实现步骤：

步骤1：构建Trie树

```
private void buildTrie() {
    for (String word : SENSITIVE_WORDS) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            node.children.putIfAbsent(c, new TrieNode());
            node = node.children.get(c);
        }
        node.isEnd = true;
        node.length = word.length();
    }
}
```

步骤2：构建失败指针（核心算法）

```
private void buildFailPointer() {
    Queue<TrieNode> queue = new LinkedList<>();

    // 根节点的子节点失败指针指向根节点
```

```
        for (TrieNode child : root.children.values()) {
            child.fail = root;
            queue.offer(child);
        }

        // BFS构建失败指针
        while (!queue.isEmpty()) {
            TrieNode current = queue.poll();

            for (Map.Entry<Character, TrieNode> entry :
current.children.entrySet()) {
                char c = entry.getKey();
                TrieNode child = entry.getValue();

                // 从当前节点的失败指针开始查找
                TrieNode failNode = current.fail;
                while (failNode != null && !failNode.children.containsKey(c))
{
                    failNode = failNode.fail; // 沿着失败指针向上跳转
                }

                // 设置失败指针
                child.fail = (failNode != null) ? failNode.children.get(c) :
root;
                queue.offer(child);
            }
        }
    }
}
```

步骤3：文本匹配与过滤

```
public String filter(String text) {
    char[] chars = text.toCharArray();
    boolean[] mask = new boolean[chars.length]; // 标记需要替换的位置

    TrieNode node = root;

    // 一次扫描匹配所有敏感词
    for (int i = 0; i < chars.length; i++) {
        char c = chars[i];

        // 沿着失败指针跳转，直到找到匹配的节点
        while (node != root && !node.children.containsKey(c)) {
            node = node.fail;
        }

        if (node.children.containsKey(c)) {
            node = node.children.get(c);
        }

        // 检查是否匹配到敏感词
        TrieNode temp = node;
```

```
        while (temp != root) {
            if (temp.isEnd) {
                // 标记需要替换的位置
                int start = i - temp.length + 1;
                for (int j = start; j <= i; j++) {
                    mask[j] = true;
                }
            }
            temp = temp.fail;
        }

        // 替换敏感词为*
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < chars.length; i++) {
            result.append(mask[i] ? '*' : chars[i]);
        }

        return result.toString();
    }
}
```

性能测试结果：

- 文本长度100字符，敏感词20个
- 平均过滤时间：<1ms
- 对消息发送延迟影响：几乎无感知

5.2.3 算法优势

- **一次扫描**：只需扫描一次文本，就能匹配所有敏感词
- **高效跳转**：失败指针实现快速跳转，避免重复匹配
- **可扩展性**：敏感词库扩展到数千个依然高效

5.3 并发安全设计

5.3.1 数据存储的线程安全

全局数据存储：

```
public class DataCenter {
    // 使用ConcurrentHashMap保证线程安全
    public static final Map<String, WebSocketSession> ONLINE_USERS =
        new ConcurrentHashMap<>();
    public static final Map<String, User> USERS =
        new ConcurrentHashMap<>();
    public static final Map<String, Group> GROUPS =
        new ConcurrentHashMap<>();
    public static final Map<String, Message> MSG_HISTORY =
        new ConcurrentHashMap<>();
}
```

选型理由：

- 并发读性能接近HashMap
- 分段锁/CAS机制保证线程安全，性能优于Hashtable
- 适合读多写少场景（聊天系统典型特征）

5.3.2 异步消息处理

线程池配置：

```
@Bean(name = "chatExecutor")
public Executor chatExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);      // 核心线程数
    executor.setMaxPoolSize(20);       // 最大线程数
    executor.setQueueCapacity(200);    // 队列容量
    executor.setRejectedExecutionHandler(
        new ThreadPoolExecutor.CallerRunsPolicy()); // 拒绝策略
    return executor;
}
```

异步处理消息发送：

```
// 对于耗时的消息发送操作，使用线程池异步处理
if ("SEND_PRIVATE".equals(request.getAction()) ||
    "SEND_GROUP".equals(request.getAction())) {
    threadPool.execute(() -> handler.handle(session, request));
}
```

设计优势：

- 提高并发能力：避免阻塞WebSocket主线程
- 资源可控：限制线程数量，防止资源耗尽
- 保证消息不丢失：使用CallerRunsPolicy拒绝策略

5.3.3 Session的线程安全

同步发送消息：

```
private void sendJson(WebSocketSession session, Object response) {
    try {
        String json = jsonMapper.writeValueAsString(response);
        if (session.isOpen()) {
            synchronized (session) { // 同步保护
                session.sendMessage(new TextMessage(json));
            }
        }
    }
}
```

```
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

同步必要性：

- WebSocket Session的sendMessage方法非线程安全
- 多线程可能并发向同一Session发送消息
- synchronized保证消息发送的原子性

5.4 权限等级划分与实现

5.4.1 权限模型设计

权限等级：

权限等级	标识	权限范围
系统管理员	ADMIN	全局禁言、强制踢出用户
群主	OWNER	群内所有管理权限（禁言、踢人、设置管理员、解散群组）
群管理员	ADMIN	群内部分管理权限（禁言、踢人）
普通用户	USER	基本聊天、好友管理功能

权限检查实现：

```
// BaseActionHandler提供权限检查方法
protected boolean isAdmin(String username) {
    User user = DataCenter.USERS.get(username);
    return user != null && user.isAdmin();
}

// Handler中的权限检查
if (!isAdmin(currentUser)) {
    sendError(session, "无权限操作：只有系统管理员可以禁言用户");
    return;
}
```

5.4.2 权限管理的业务逻辑

用户模块权限检查：

```
// UserServiceImpl中的权限检查示例
public boolean muteGroupUser(String operator, String groupId,
                             String targetUserId, long durationMillis) {
```

```
Group group = DataCenter.GROUPS.get(groupId);

// 检查权限
if (!isGroupOwnerOrAdmin(groupId, operator)) {
    throw new SecurityException("无权限操作: 只有群主或管理员可以禁言");
}

// 权限细化检查
if (isGroupOwner(targetUserId, groupId)) {
    throw new SecurityException("不能禁言群主");
}

if (isGroupAdmin(targetUserId, groupId) && !isGroupOwner(operator, groupId)) {
    throw new SecurityException("管理员不能禁言其他管理员");
}

// 执行禁言操作
// ...
}
```

设计特点:

- **细粒度权限控制**: 不同操作有不同权限要求
- **权限继承关系清晰**: 群主 > 管理员 > 普通用户
- **防止权限滥用**: 权限检查在Service层和Handler层双重验证

5.5 模板方法模式的应用

5.5.1 BaseActionHandler设计

实现代码复用与统一错误处理。

代码实现:

```
public abstract class BaseActionHandler implements ActionHandler {
    @Autowired
    protected ObjectMapper objectMapper;

    // 模板方法: 发送响应
    protected void sendResponse(WebSocketSession session, WsResponse response) {
        if (session == null || !session.isOpen()) return;
        try {
            String json = objectMapper.writeValueAsString(response);
            session.sendMessage(new TextMessage(json));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
// 模板方法：发送错误
protected void sendError(WebSocketSession session, String errorMsg) {
    sendResponse(session, WsResponse.error(errorMsg));
}

// 模板方法：发送成功响应
protected void sendSuccess(WebSocketSession session, Object data) {
    sendResponse(session, WsResponse.builder()
        .type("SUCCESS")
        .data(data)
        .build());
}

// 辅助方法：获取当前用户
protected String getCurrentUser(WebSocketSession session) {
    return (String) session.getAttributes().get("username");
}
}
```

优势：

- **代码复用**：25个Handler共享公共方法，减少重复代码
- **统一接口**：所有Handler使用相同的错误处理和响应格式
- **易于维护**：修改公共逻辑只需修改基类

5.6 消息撤回机制设计

5.6.1 时间限制实现

```
private static final long RECALL_TIME_LIMIT = 2 * 60 * 1000; // 2分钟

public boolean recallMessage(String msgId, String operator) {
    Message message = DataCenter.MSG_HISTORY.get(msgId);
    if (message == null) {
        return false;
    }

    // 检查时间限制
    long timeElapsed = System.currentTimeMillis() -
message.getTimestamp();
    if (timeElapsed > RECALL_TIME_LIMIT) {
        return false; // 超过2分钟，无法撤回
    }

    // 检查权限：只能撤回自己发送的消息
    if (!message.getFromUser().equals(operator)) {
        return false;
    }

    // 执行撤回：从历史记录中删除
    DataCenter.MSG_HISTORY.remove(msgId);
}
```

```
// 通知所有相关用户
broadcastRecallEvent(message, operator);

return true;
}
```

设计考虑：

- **时间限制：**参考微信、QQ等主流聊天工具的做法
- **权限检查：**只能撤回自己发送的消息
- **实时同步：**撤回后立即通知所有相关用户更新UI

5.7 @提醒功能实现

5.7.1 前后端协作实现

前端识别@符号：

```
// 前端识别@符号，解析出被@的用户
handleAtInput(input) {
    const text = input.value;
    const atMatch = text.match(/@(\w+)/g);
    if (atMatch) {
        const atUsers = atMatch.map(m => m.substring(1)); // 提取用户名
        return atUsers;
    }
    return [];
}
```

后端存储@信息：

```
// Message对象包含atUsers字段
Message msg = Message.builder()
    .fromUser(fromUser)
    .toUser(toUser)
    .content(content)
    .atUsers(atUsers) // 存储被@的用户列表
    .build();
```

后端推送提醒：

```
// 发送消息时，检查接收者是否被@
if (message.getAtUsers().contains(receiver)) {
    // 发送@提醒事件
    sendAtMentionEvent(receiver, message);
}
```

前端显示提醒：

```
// 收到@提醒事件
case 'EVENT_AT_MENTION':
    showAtMentionBadge(message.groupId);
    break;
```

设计优势：

- 前后端分离：前端负责解析，后端负责存储和推送
- 用户体验好：@提醒实时显示，点击后自动清除
- 可扩展：支持@多个用户，支持@所有人（仅群主）

6. 系统测试与验证

6.1 功能测试

6.1.1 用户认证测试

测试用例：

1. 新用户首次登录自动注册
2. 已注册用户密码验证
3. 密码错误拒绝登录
4. 管理员账号自动识别
5. 多端登录顶号功能

测试方法：

- 使用浏览器多窗口模拟多用户场景
- 检查服务器日志确认流程正确

6.1.2 消息功能测试

单聊测试：

1. 发送私聊消息
2. 接收方实时收到消息
3. 离线用户消息保存
4. 历史消息查询

群聊测试：

1. 发送群聊消息，所有成员收到
2. @提醒功能正常
3. 群成员上线后收到历史消息

消息撤回测试：

1. 2分钟内可以撤回
2. 超过2分钟无法撤回
3. 撤回后所有端同步删除
4. 只能撤回自己发送的消息

消息反应测试：

1. 添加反应（点赞、点踩等）
2. 取消反应
3. 反应计数实时更新

6.1.3 权限管理测试**管理员权限：**

1. 全局禁言功能
2. 强制踢出用户
3. 管理员标识显示

群主权限：

1. 设置/取消管理员
2. 禁言/解禁成员
3. 踢出成员
4. 修改群名称
5. 解散群组

群管理员权限：

1. 禁言/解禁成员
2. 踢出成员
3. 不能设置管理员
4. 不能解散群组

6.1.4 好友管理测试

1. 发送好友申请
2. 接收好友申请通知
3. 接受/拒绝好友申请
4. 双方好友列表同步更新
5. 删除好友功能

6.1.5 敏感词过滤测试

1. 单个敏感词过滤
2. 多个敏感词过滤
3. 敏感词在句子中间
4. 英文敏感词过滤

5. 过滤后日志记录

6.3 兼容性测试

浏览器兼容性：

- Chrome 90+：完全支持
- Firefox 88+：完全支持
- Safari 14+：完全支持
- Edge 90+：完全支持

功能兼容性：

- WebSocket连接正常
- JSON解析正常
- 所有功能在所有浏览器中正常工作

7. 参考资料与致谢

7.1 技术参考资料

- Spring Boot官方文档：<https://spring.io/projects/spring-boot>
- WebSocket协议规范：RFC 6455
- AC自动机算法：Aho-Corasick Algorithm
- Spring WebSocket文档：<https://docs.spring.io/spring-framework/reference/web/websocket.html>

7.2 项目资源

- GitHub仓库：https://github.com/Ryllis-y/OO_FinalProject_Chat

附录

附录A：API接口列表

主要接口列表：

Action	功能	必需参数	权限要求	响应类型
LOGIN	登录/注册	username, password	无	LOGIN_RESP
SEND_PRIVATE	发送私聊消息	targetUser, content	已登录	EVENT_CHAT_MSG
SEND_GROUP	发送群聊消息	groupId, content	已登录，群成员	EVENT_CHAT_MSG
RECALL_MSG	撤回消息	msgId	发送者	EVENT_MSG_RECALLED

Action	功能	必需参数	权限要求	响应类型
MSG_REACT	消息反应	msgId, reactType	已登录	EVENT_MSG_REACT
GET_HISTORY	获取历史消息	targetUser/groupId, beforeTime(可选)	已登录	SUCCESS
GET_GROUPS	获取群组列表	无	已登录	SUCCESS
CREATE_GROUP	创建群组	groupName, members(可选)	已登录	GROUP_CREATED
JOIN_GROUP	加入群组	groupId	已登录	GROUP_JOINED
LEAVE_GROUP	退出群组	groupId	已登录, 群成员	SUCCESS
DISSOLVE_GROUP	解散群组	groupId	群主	GROUP DISSOLVED
GET_GROUP_MEMBERS	获取群成员	groupId	已登录, 群成员	SUCCESS
SEND_FRIEND_REQUEST	发送好友申请	toUser	已登录	SUCCESS
ACCEPT_FRIEND_REQUEST	接受好友申请	fromUser	已登录	SUCCESS
REJECT_FRIEND_REQUEST	拒绝好友申请	fromUser	已登录	SUCCESS
GET_FRIENDS	获取好友列表	无	已登录	SUCCESS
GET_FRIEND_REQUESTS	获取好友申请列表	无	已登录	SUCCESS
DELETE_FRIEND	删除好友	friendId	已登录	SUCCESS
MUTE_USER	全局禁言	targetUser, duration	系统管理员	SUCCESS
KICK_USER	强制踢出	targetUser	系统管理员	SUCCESS

Action	功能	必需参数	权限要求	响应类型
MUTE_GROUP_USER	群内禁言	groupId, targetUser, duration	群主/管理员	SUCCESS
KICK_GROUP_USER	踢出群成员	groupId, targetUser	群主/管理员	GROUP_KICKED
SET_GROUP_ADMIN	设置管理员	groupId, targetUser, isSet	群主	SUCCESS
GET_ONLINE	获取在线用户	无	已登录	ONLINE_LIST
HEARTBEAT	心跳包	无	已登录	SUCCESS

请求格式：

```
{
  "action": "ACTION_NAME",
  "params": {
    // 具体参数
  }
}
```

响应格式：

```
{
  "type": "RESPONSE_TYPE",
  "code": 200,
  "msg": "ok",
  "data": {
    // 业务数据
  }
}
```

错误响应格式：

```
{
  "type": "ERROR",
  "code": 500,
  "msg": "错误信息",
  "data": null
}
```

- 后端代码：约3500行Java代码
- 前端代码：约2500行JavaScript代码
- HTML/CSS：约800行
- Handler数量：25个
- API接口：30+个

Directory /Users/ryllis/IdeaProjects/chat

Total : 78 files, 15119 codes, 1006 comments, 4041 blanks, all 20166 lines

Summary / [Details](#) / [Diff Summary](#) / [Diff Details](#)

Languages

language	files	code	comment	blank	total
Markdown	24	8,481	15	2,746	11,242
Java	44	2,976	587	759	4,322
JavaScript	1	1,929	310	269	2,508
PostCSS	1	883	14	156	1,053
HTML	2	449	21	47	517

附录C：开发时间线

阶段	时间	主要工作
需求分析	11.25-11.30	架构设计、接口定义
并行开发	12.1-12.15	各模块功能开发
功能迭代	12.16-12.21	代码重构、Bug修复、优化
测试与文档	12.22-12.24	全面测试、文档编写

编写团队：#1f1e33